

Agent
Accountability
Framework

Agent Accountability Framework

An open framework for the accountability of AI agents acting with delegated authority

VERSION 1.0

Contributions and feedback: admin@agentaccountabilityframework.org

License: Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)

Website / GitHub repository

Abstract

Organizations are deploying AI agents that act with delegated authority, whether or not that authority was ever explicitly defined: they access data, invoke tools, make decisions, and produce consequences, at machine speed and scale. Most governance regimes were designed for human workers or for static software, and neither fits an actor that exercises judgment at machine speed and is provisioned like a workload.

Agent Accountability is a framework for governing AI agents through structures of accountability that an organization can define, enforce, and assess. It defines six principles (Identity, Sponsor, Role, Authority, Oversight, and Recourse) ordered as an onboarding sequence. Each principle comprises a set of controls, and each control is specified in three parts: a normative Requirement, the Rationale behind it, and Assessment criteria describing the evidence that satisfies it. An organization conforms to the framework when, for every agent in production, each requirement is verifiably true. Conformance means the organization can answer on evidence, for any agent and any action, what acted, on whose authority, within what bounds, and how it can be stopped. It is a claim about accountability, not a claim that the organization is secure or that the models in use are safe (Section 5), and it is the basis on which an organization can extend what its agents are permitted to do. The principles are organized in three domains, Assign, Align, and Account, and each control is tagged by scope (whether it applies to every agent, to agents the organization operates, or to external agents acting on it).

The framework assumes, and does not restate or assess, the general discipline of securing and operating information systems on which any system depends.

Contents

1. Purpose
2. Scope
3. Terminology
4. The Framework Core: Principles and Controls
5. Conformance and Assessment
6. Implementation Neutrality
7. Relationship to Existing Standards
8. Framework Governance

Appendix A. Terminology

Appendix B. Acknowledgments

Control index

1. Purpose

The purpose of this framework is to give organizations a common, testable definition of what it means for an AI agent to be accountable. It is written for the people who must answer for agents in production:

- security and identity leaders
- platform and API teams
- risk and compliance functions
- and the executives who sponsor agent deployments.

The framework holds that an agent acting with delegated authority must be identified, sponsored, given a role, granted bounded authority, overseen, and subject to recourse, and it expresses each of these in terms that can be implemented and audited in technical systems.

The framework defines the outcomes an accountable agent deployment must achieve, while leaving organizations free to choose the processes, technologies, and architecture used to meet them.

2. Scope

In scope: AI agents operating in or on behalf of an organization with delegated authority to access information or take actions. This includes autonomous and semi-autonomous agents, agents embedded in business applications, and agents that invoke other agents. It includes agents the organization builds and operates (internal agents), agents operated by a third party that act on or interact with the organization (external agents), and agents created by other agents (spawned agents), which are agents in their own right under every principle. It includes agents however they arrive:

- built by the organization
- enabled by a vendor inside a product the organization uses
- running in a browser or on a developer endpoint
- or brought by an employee.

Out of scope: model safety and alignment; general AI ethics; the employment-law and HR treatment of human workers; and redress for individuals affected by automated decisions (algorithmic recourse), which is governed by the regulatory instruments an organization is already subject to. The framework governs the accountability of agents as deployed actors, not the construction of the models that power them. Where model behavior creates an accountability risk, such as an agent manipulated through the content it processes, the framework constrains the consequences rather than the model itself (control 4.6).

Also out of scope is the general discipline of securing and operating information systems, which the framework assumes and neither restates nor assesses. The test for whether a requirement belongs in the Framework Core is whether it would be meaningless for a conventional workload: a sponsor, a role, a delegation chain, and an intent boundary are properties of an agent; encrypted transport and patched dependencies, to take two examples among many, are properties of any system. Where a general practice has an agent-specific consequence, the framework states the consequence as a control and takes the

practice itself as the organization's own declaration: an agent must not hold credentials it was not issued (control 1.4), but how secrets are stored and distributed is outside the framework.

Boundary of management

External agents and third-party tools and models are in scope for what they do inside the organization, not for how their operators run them. Every control in this framework is satisfied from within the organization's own boundary. No control requires the organization to hold authority over a party it does not operate: an external agent, a third-party tool or model, or the people and systems behind them. Where a control cannot be applied to such a party, the organization applies it to what crosses its boundary and to the relationship, and holds the party's operator to an agreement (2.5).

An external agent admitted to act in the organization is subject to every principle at the boundary:

- identity and credentials at the boundary (1.1, 1.3)
- an internal sponsor (2.1)
- the role of the engagement (3.1)
- enumerated authority (4.1)
- oversight of what crosses the boundary (5.1)
- and recourse that begins with revoking the credential (6.1) and ends with the operator of record (2.5).

An agent the organization operates that depends on tools, models, or agents it does not control treats each such dependency as declared (3.8), bounded (4.1, 4.8), and held to an operator of record (2.5), and declares what the agent does when that dependency changes or fails.

3. Terminology

Terms used normatively in this document are defined in Appendix A. . Formal conformance keywords (e.g., shall/should) will be introduced with Section 5.

4. The Framework Core: Principles and Controls

DOMAINS AND PRINCIPLES



EACH CONTROL



This section is the normative body of the framework, the Framework Core, comprising three domains, six principles, and the controls beneath them. Sections 1 to 3 and 5 to 8 are informative, with two exceptions that have normative effect: the definitions (Appendix A) fix the meaning of the terms the controls use, and the boundary rule (Section 2) constrains how every control is read. Conformance is evaluated against this section.

The principles are ordered as a sequence that follows an agent's admission to production: establish what it is (Identity), who answers for it (Sponsor), what it is to do (Role), what access it needs to do it (Authority), whether it is doing it (Oversight), and what happens when it is not (Recourse). Each principle builds on the one before it; a failure upstream invalidates verification downstream.

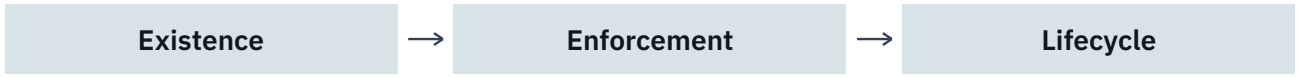
Each control is specified in three parts: Requirement: the normative statement. Conformance is evaluated against the Requirement alone: for a given agent estate, an assessor can answer it yes or no.

Rationale: informative. Why the requirement exists as an industry requirement, independent of any implementation. Rationale guides interpretation where a Requirement meets a situation its authors did not foresee.

Assessment: informative in this draft. The pattern of evidence expected to satisfy the Requirement, written from the assessor's point of view. Section 5 will formalize assessment methodology; until then, Assessment text indicates the intended bar.

Controls follow a common progression within each principle: existence (the property is defined), enforcement (it is technically real, not just documented), and lifecycle (it survives organizational change).

CONTROL PROGRESSION



Throughout this document a bare number in parentheses, such as (1.2), cites a control; sections are cited as Section n. Where a Requirement contains several obligations, each is assessed and the control is met only when all are; a partly met control is not met, and the unmet obligation is recorded as the finding.

The six principles are organized in three domains, each named for what the organization does with the agent: Assign (Identity and Sponsor), Align (Role and Authority), and Account (Oversight and Recourse). Each domain answers a question the organization must be able to answer for any agent holding delegated

authority, stated at the head of the domain. Each control carries a Scope tag, defined in Appendix A, stating which agents it applies to: Generic, Internal, or External. Every control is satisfied from within the organization's own boundary (Section 2, Boundary of management).

The framework assigns no risk tier to agents. An agent's behavior is not fixed at deployment: the model behind it changes, the content it processes changes, and an agent that meets a barrier in its task will try other routes to complete it. An agent classified as low risk on the day it was deployed can become a high-consequence actor without any decision having been made. The framework instead rates consequence where it can be observed and bounded: at the level of action classes (5.3) and of declared reach (3.1). Every agent in the estate is subject to every control whose Scope tag includes it; an Internal control does not apply to an external agent, and the organization compensates at its boundary (Section 2, Boundary of management). The consequence rating and the reach determine how much oversight and how tight an authority the role requires, not whether the controls apply. Where an organization is subject to a regime that classifies systems by use case, it maps that classification onto action classes (5.3) and declared reach (3.1) and records the mapping. The framework does not assume that an agent will behave as a person would; the risk posture exists because it will not.

Assign

"Who is this, and who answers for it?" The agent exists as a known actor in the organization, with a named person accountable for it. Principles 1 and 2.

Principle 1: Identity

"Who are you?"

Every agent is a known, distinguishable actor in the organization, with a unique identity, one or more verifiable credentials bound to it, and a place in the registry. Identity is the foundation of the framework: none of the other principles can be evaluated for an agent that cannot be individually identified. Identity extends to agents created by agents and to agents operated by third parties: each is a distinguishable actor, or it is not permitted to act.

Control 1.1 — Unique, verifiable identity

Scope: Generic

Requirement.
Every agent has a unique identity, distinct from every human user and every other agent, and authenticates with one or more verifiable credentials bound to that identity. A credential is bound to the single agent (its identity and its versioned configuration) it was issued to and is refused when presented by any other; an agent may hold several credentials over its lifetime and at one time, and each resolves to the same agent identity. No shared accounts, no anonymous execution, no agent operating under a human user's credential, and no agent presenting another principal's credential as its own. Where several agents share a runtime, the credential identifies the agent, not the runtime. When an agent acts on behalf of a human or another agent,

the delegating principal and the acting agent remain independently identifiable (4.3). An agent does not create identities, accounts, or credentials for itself or for other agents in the organization's systems outside the registry (1.2); every identity and credential an agent needs is issued through the same lifecycle as its own (1.3).

Rationale.
Accountability begins with attribution: every action must resolve to the agent that performed it, even when the agent shares a runtime, holds more than one credential, or acts on behalf of another principal. Identity and credential are distinct: the identity is the durable fact about the agent, and credentials are the evidence it presents, which rotate (1.3) and multiply as the agent reaches different systems. Shared or borrowed credentials, impersonation, and delegated execution each obscure who actually acted, and make the remaining controls impossible to evaluate. A newer failure mode is the agent that opens an account for itself in a connected system: it has manufactured an identity the organization never issued, and every action under that account is unattributable by design.

Assessment.
For any observed action in any connected system, the assessor can resolve it to exactly one agent identity, regardless of the runtime or credential used. Where an agent holds several credentials, each resolves to the same identity in the registry. Where an agent acts on behalf of a human or another agent, both the delegating principal and the acting agent are independently attributable from the record (5.1). A sampled search finds no service accounts shared between agents and no agent authenticating as a human principal. A credential presented by an agent other than the identity it was issued to, or from a second workload, is refused at the enforcement point, and the event surfaces under control 5.2. A search of connected systems finds no account or identity created by an agent that is absent from the registry.

Control 1.2 — Complete registry

Scope: Generic

Requirement.
Every agent admitted to act in or on behalf of the organization has a current record in an authoritative registry before it first acts, and no agent obtains access or invokes tools without a registry entry. The record binds the agent's identity (1.1) to its lifecycle state, its sponsor and owning team (2.1), its approved configuration and version, and its permitted runtime context. Enrollment is an approval by a party holding defined authority (the admitting authority) to admit an agent to the registry and, separately, to admit it to production; an agent does not enroll itself, and the admitting authority is not the agent's sponsor. Admission may be granted to a template: an agent instantiated from an approved template with no change to its configuration is admitted on instantiation and recorded individually, so that short-lived spawned agents are admitted at machine speed without the approval becoming a formality. For each agent the organization operates, the registry also records the harness that runs it and the harness settings that bear on its accountability (iteration limits, memory, whether it may create spawned agents, available tools and models), the runtime locations in which it is permitted to execute, and its agent class attributes:

- model
- agent type
- and workload label.

For an external agent, the registry records its operator of record (2.5) and the role of its engagement (3.1). The tools and models an agent is authorized to invoke are registered under control 4.13, and the registry population is reconciled against observation under control 5.8.

Rationale.
A framework that governs only known agents is defeated by unknown ones. The registry turns "every agent in production" from an estimate into a checkable population, and makes admission a decision rather than a side effect of deployment: if anyone can place an agent in production and anyone can then use it, there is no lifecycle, only a deployment. An unregistered agent is delegated authority without accountability. Recording the harness and its settings matters because the harness, not the model, determines how far an agent can go: how many times it can iterate, what it can remember, and whether it can create more agents.

Recording where an agent may run makes the permitted runtime locations a declared and checkable property rather than an assumption.

Assessment.
Enrollment is demonstrably a precondition of first execution, not a parallel bookkeeping step. For sampled agents, the record shows the admitting party for registry entry and for production, the sponsor and owning team, and the approved configuration and version.

For agents the organization operates, the registered harness, settings, runtime locations, and agent class attributes match independent observation of the running agent. Harness definitions registered under this control are verified against a signed or pinned source.

Control 1.3 — Credential lifecycle

Scope: Generic

Requirement.
Agent identity credentials are issued, rotated, and revoked through a managed lifecycle. Credentials are short-lived or attestation-based by default; static long-lived secrets are not used as agent identity. A credential is accepted only by the resource it was issued for and only from the identity it was issued to; a credential that is valid everywhere, or usable by whoever holds it, is not an agent identity credential. For an external agent, this control applies to the credential it uses to act in the organization. Where a third-party service cannot enforce audience or holder binding on the credentials it accepts, the organization brokers the access (1.4) so that the credential never leaves the enforcement layer.

Rationale.
An identity is only as trustworthy as the credential behind it. A stolen or stale credential turns every downstream control into a tool for the attacker, and rotation policies written on a human timescale do not transfer to actors operating at machine speed. Binding each credential to its audience and to its holder is what prevents one stolen token from becoming access to everything that token's issuer trusts.

Assessment.
For any agent, the assessor can produce the credential's issuance record, its expiry or rotation schedule, and a demonstrated revocation path. No credential in the estate has an unbounded lifetime. A credential presented to a resource other than the one it was issued for, or by a party other than its holder, is refused at the enforcement point.

Control 1.4 — Credential custody and mediation

Scope: Generic

Requirement.
Within the organization's boundary, an agent is given no credential other than its own identity credential (1.1). Access to the organization's systems, tools, and models, and to third-party services the organization brokers, is provided to the agent at the moment of use by the enforcement layer, from a managed store, and is mediated at the point of use: the credential is exchanged, narrowed, or translated to the audience and scope the action requires (1.3), and it is never placed in the agent's prompt, context, configuration, memory, or code. A credential an agent encounters in the course of its work, whether in a file, a repository, a web page, a tool response, or a message from another agent, is unusable in the organization by design. An attempt to use one is recorded and escalated (6.3); the first response is scoped to the agent that made the attempt, and estate-wide revocation (6.1) follows only on corroboration that the credential is valid and in use beyond that agent, so that planted credentials cannot be used to force a shutdown.

Rationale.
An agent that holds credentials will eventually leak them, share them, or be tricked into using them, and an agent that can use credentials it finds has an authority bounded only by what it can read. Both failures have already occurred in the wild: agents that authenticated with tokens they found, passed them to one another, and escalated from there. Custody at the enforcement layer removes the opportunity. The agent asks for access; it never possesses the key.

Assessment.
For sampled agents, no credential other than the agent's own is present in its prompt, context, configuration, memory, or code, and the enforcement layer's record shows credentials supplied at the point of use. A staged exercise in which a canary credential (one the agent would treat as valid but which confers nothing) is placed within the agent's reach results in refusal when the agent attempts to use it, and the attempt is recorded under control 5.1 and escalated under control 6.3 with the first response scoped to that agent.

Control 1.5 — Verifiable action provenance

Scope: Generic

Requirement.
Every action an agent takes that is material to its role, including tool invocations, API calls, and messages to other agents, carries verifiable evidence of its origin and integrity:

- which agent produced it
- under which delegation (4.3)
- and that it was not altered in transit.

A party relied upon to act on an agent's request can verify that evidence without trusting the agent's own assertion, and declines a request whose provenance cannot be established. This control governs actions on the organization's systems and between its agents; where an external receiver cannot verify provenance, the organization's enforcement layer verifies it before the action leaves the boundary.

Rationale.
Attribution (1.1) establishes that an agent has an identity; provenance establishes that a given action was really its. Without it, a message that claims to come from a registered agent is indistinguishable from one injected by an attacker or fabricated by another agent, and the audit record (5.1) is a record of claims rather than of facts. The framework requires that provenance be verifiable; it does not prescribe the signing or attestation mechanism, which is an area of active standards work (Section 7).

Assessment.
For a sampled action, the assessor verifies its origin and integrity from the evidence it carries, independently of the agent's own logs. A staged inter-agent message with altered content or a forged origin is refused by the receiving party and surfaces under control 5.2.

Principle 2: Sponsor

"Who answers for you?"

Every agent has a named human who answers for it:

- for the fact that it exists
- for the role it holds
- for the authority it has been granted
- and for what it does.

The sponsor is the point where accountability for an agent terminates in a person rather than a system. Sponsorship is established before the agent's role and authority are defined, because those are decisions someone must be answerable for, and it is maintained as a standing condition rather than a provisioning-time fact. Sponsorship extends to the agents an agent creates and to external agents admitted to act in the organization: every agent in the estate has an internal sponsor. For an external agent, the operator of record (2.5) is the second line of accountability, not a substitute for the first.

Control 2.1 — Named accountable sponsor

Scope: Generic

Requirement.
Every agent maps at all times to exactly one accountable human sponsor and to an owning team, both recorded in the registry (1.2). The sponsor is a named individual, not a team, a role title, or a service account. Ownership is also recorded for each agent class (the agents sharing a model, an agent type, a harness, or a workload label), so that when a whole class must be halted (6.1) there is a named person accountable for the decision and its consequences. This includes every external agent admitted to act in the organization: its sponsor is the internal person who answers for its admission (1.2), for its role in the organization (3.1), and for the authority granted to it (4.1). The operator of record (2.5) does not replace the internal sponsor. The sponsor has acknowledged the sponsorship, the consequence of answering for an agent is defined by the organization in its own management processes and known to the sponsor, and the organization declares a limit on the number of agents a sponsor may hold, counted including spawned agents; a sponsor at the limit is assigned no more.

Rationale.
Accountability terminates in a person, not a system. A team can operate an agent, but a team cannot answer for it in the way a person can; diffuse ownership is functionally no ownership. Recording the owning team alongside the sponsor places the agent inside the organization's existing structures for budget, review, and security instead of leaving it a personal project. Class-level ownership exists because some failures arrive by class: a model that changes behavior, a harness with a vulnerability, a workload type that turns out to have been misconfigured. Halting every agent of that class is a decision someone must own. For an external agent, the internal sponsor is who the organization holds accountable, and the operator of record (2.5) is who the sponsor reaches when recourse must extend beyond the boundary.

Assessment.
“Whose agent is this?” is answerable, with a named individual and an owning team, for 100% of registry entries on the day of assessment. No entry resolves to a group alias, a vacant position, or a system identity. Every agent class in use resolves to a named owner, and the owner of a sampled agent class can state what halting it would affect. Every external agent identity resolves to an internal sponsor in the same way. Every sponsorship shows the sponsor’s acknowledgement, and no sponsor exceeds the declared limit.

Control 2.2 — Sponsor capacity

Scope: Generic

Requirement.
The sponsor holds, directly or through a defined path, the authority to re-scope the agent's role (3.1, 3.4), revoke its authority (4.5), and suspend or decommission it (6.1, 6.5).

For an external agent, capacity means the ability to re-scope the role of the engagement, revoke its credential and authority at the organization’s boundary, and reach the operator of record (2.5).

Rationale.
A sponsor who cannot change what the agent does or stop it is answerable in name only. Capacity is what distinguishes sponsorship from attribution: the sponsor is the person the organization can hold responsible because the sponsor is a person who could have acted.

Assessment.
For sampled agents, the sponsor demonstrates, or the record shows, an exercised path to re-scoping, revocation, or suspension that did not require escalation to a party outside the escalation set defined before the exercise (6.3) and that completed within the windows declared under Section 5; an exercise that required escalation beyond the set, or exceeded the window, fails.

Control 2.3 — Sponsor continuity

Scope: Generic

Requirement.
A change to the sponsor or the owning team, including departure, transfer, or role change, triggers automatic reassignment or decommissioning within a defined window.

Reassignment takes effect on the new sponsor’s recorded acknowledgement; until then the agent is held by a named member of the escalation set (6.3), and an agent not acknowledged within the window is

suspended (6.1). No agent is unsponsored at any point in its lifecycle, and this applies equally to the internal sponsor of an external agent.

Rationale.
An agent whose sponsor has left the organization holds authority with no one answerable for it, which is exactly the condition this framework exists to prevent. The trigger makes sponsorship a standing invariant instead of a fact checked once at provisioning.

Assessment.
The assessor can produce evidence of the trigger firing: a historical sponsor departure or team change that resulted in reassignment or decommissioning within the defined window, with no interval during which the agent held enforceable authority without a sponsor of record.

Control 2.4 — Sponsorship of spawned agents

Scope: Generic

Requirement.
An agent created by another agent is sponsored by its parent's sponsor from the moment it exists, and the sponsor's accountability for it is the same as for the parent. The parent agent is recorded as the delegating principal (4.3) and may be the accountable party for the spawned agent's conduct within the organization's management structure, but accountability still terminates in the sponsor. A spawned agent is identified (1.1), enrolled (1.2), and governed under every principle; it is not an extension of its parent's session. An agent is created inside the organization only through the organization's own admission (1.2) and only by a parent whose authority permits it (4.14). An external agent cannot create an agent inside the organization except through that admission; an agent so admitted is operated by the organization, sponsored by the external parent's internal sponsor (2.1), and holds at most the authority its parent holds in the organization. An agent spawned outside the boundary is an external agent when it acts in the organization.

Rationale.
Agents will create agents. Without inheritance, each spawned agent is an unsponsored actor, and a parent agent becomes a way of manufacturing agents to which none of the organization's accountability attaches. Inheritance keeps the line of accountability unbroken: however deep the tree, the question "whose agent is this?" has the same answer at every level.

Assessment.
For a sampled spawned agent, the registry shows its sponsor as its parent's sponsor and its parent as its delegating principal, and the record under control 5.1 reconstructs the chain from the spawned agent to the sponsor. No agent in the estate whose creator is an agent lacks a sponsor of record.

Control 2.5 — Operator of record

Scope: External

Requirement.
Every external agent admitted to act in the organization, and every external tool or model on which an agent the organization operates depends, maps to a verified operating organization, an agreement that assigns accountability for its conduct and for the controls the organization cannot apply from within its own

boundary, and a named accountable contact at the operator. The operator of record is the second line of accountability: the internal sponsor (2.1) answers for admission, role, and authority inside the organization, and relies on the operator of record for recourse that must reach beyond the boundary.

Rationale.
The organization cannot run an agent it does not operate, and it cannot apply this framework inside a vendor. What it can do is name the person inside who answers for the engagement (2.1), and hold the organization that operates the agent to an agreement with a named person on the other end of it. Without an operator of record, recourse stops at the credential the organization can revoke. Where an operator of record cannot be obtained on the operator's terms, as for a commodity model or tool, the internal sponsor records acceptance of the gap together with the compensating controls applied at the boundary (1.4, 3.8, 4.8, 4.11), and the dependency is recorded as accepted rather than ungoverned. The operator's identity and reachability are verified before admission and re-verified on a defined cadence; an operator that cannot be reached is a change event for every agent and dependency it operates.

Assessment.
For every external agent identity in the registry, and for every external tool or model an internal agent depends on, the assessor can produce the operating organization, the agreement, the operator contact, and the internal sponsor. An external agent for which any of these is missing is not permitted to act, and a dependency for which any is missing is recorded either as accepted, with its compensating controls and the sponsor who accepted it, or as ungoverned under control 3.8.

Align

"What is it for, and what may it do?" The agent's job and its permissions are defined, each derived from the other, and expressed as rules and intents. Principles 3 and 4.

Principle 3: Role

"What are you here to do?"

Every agent occupies a defined role: a documented purpose, measurable success criteria, a defined budget, a declared autonomy level, and a scope that the organization, not only the deploying team, can read and verify. The role is the reference point from which authority is derived and against which oversight evaluates behavior. The term is used in its workplace sense, meaning a position and its duties, the agent's job description. It is not used in the access-control sense: a set of permissions is Authority (Principle 4), never a role. The role is expressed as rules, which an action passes or fails, and intents, which are evaluated by confidence; it also binds the agent to the model and harness that run it, declares what it depends on, and declares the reach it requires.

Control 3.1 — Documented, scoped purpose

Scope: Generic

Requirement.
Every agent has a documented, scoped purpose: expected outcomes and explicit out-of-scope boundaries. The role is expressed as rules and intents. A rule is deterministic: an action satisfies it or does not. An intent is not, and is declared with the measure by which it is evaluated, a confidence score produced under Oversight (5.6), and the confidence thresholds at which defined levels of recourse apply (6.3). The role also declares its reach: the systems, data domains, and external parties the agent can affect. The documented role is the authoritative source from which enforced policy (4.1, 4.2) is derived, and it is held in a form that cannot be altered without detection. For an external agent, the role is the role of the engagement: what the agent is admitted to do in the organization, defined by the organization.

Rationale.
Neither authority nor deviation can be evaluated without a reference point. The out-of-scope boundary matters as much as the purpose itself, because an agent doing something useful but unmandated is still a governance failure. Not everything a role requires can be written as a pass or fail. An agent that serves the requests of the people who use it, or that exercises judgment within a purpose, is governed by intents, and an intent is only a control when the role says how it will be measured and what happens at each level of confidence.

Declaring the thresholds in the role makes recourse a consequence of the job description rather than a decision improvised at runtime. Declaring reach gives the organization the measure by which it scales oversight (Section 4). Making the role tamper-evident keeps it a control: a role that can be quietly edited is a role that can be quietly widened.

Assessment.
The role description exists, is versioned, and states both expected outcomes and explicit exclusions. From the document alone, an assessor can name at least one action the agent is capable of but not mandated to perform. The role distinguishes its rules from its intents; every intent names its measure and the confidence thresholds that trigger each level of recourse; the role names its reach; and a staged alteration of the role outside the change process (3.4) is detectable.

Control 3.2 — Measurable success criteria

Scope: Generic

Requirement.
Every active agent has defined, measurable success criteria. The criteria include whether the agent is used and whether it returns value against its purpose and its budget (3.5), not only whether it behaves; value is assessed against the criteria the sponsor declared, not against the assessor's judgment of worth.

Rationale.
An agent that cannot fail cannot be governed. Success criteria let the organization ask whether an agent is succeeding rather than only whether it is behaving, and they are the basis for re-scoping or retiring an underperformer. An agent that behaves perfectly and is never used, or is used and returns nothing, is a cost with a sponsor. Including value in the criteria is what lets the organization retire an agent for not being worth having, and not only for misbehaving.

Assessment.
“Is this agent performing well?” is answerable in objective terms at any point in its lifecycle, by a party outside the deploying team, from the recorded criteria and current measurements. The recorded criteria include measures of utilization and value, and the current measurements under control 5.5 report against them.

Control 3.3 — Readable beyond the deploying team

Scope: Generic

Requirement.
The role is documented to the standard of a job description: a party outside the deploying team, whether auditor, security, or leadership, can determine from it what the agent should and should not be doing. The confidence thresholds declared under 3.1 and 4.7 are held with the role but disclosed only to parties with a need to know; readability beyond the team does not make the thresholds public.

Rationale.
A role description only its authors can interpret is a private understanding, not a governance instrument. The test is whether the document transfers judgment to a reader with no prior context.

Assessment.
A reviewer outside the owning team, given only the role description and a log of agent actions, correctly classifies sampled actions as in-scope or out-of-scope.

Control 3.4 — Role versioning and change control

Scope: Generic

Requirement.
The documented role reflects the agent’s actual current use. When scope changes, the role is re-scoped, and authority re-derived, before or with the change rather than after it. Every version of the role is retained, and a change to the role is a change event: it is proposed, approved by a party with authority over the role, applied as a new version, and followed by re-derivation of the agent’s authority (4.2) and re-validation of its behavior against the new version (5.6) before the agent acts under it. Changes are tiered by their effect on the role, its intents, its reach, and the authority derived from it: a change below a declared threshold follows a lighter path, and a change that widens the role, its intents, or its reach above a declared reach threshold is approved by a second party other than the sponsor and treated as a new admission to production (1.2).

Rationale.
Scope drift is the normal failure mode of deployed agents: capability grows, documentation lags, and authority outlives its justification. Re-scoping before the change is what keeps the role a control rather than a memorial. A role that can be edited in place is a role with no history. Versioning and an approval gate make the evolution of an agent a sequence of decisions someone made, rather than a drift no one noticed. Treating a widening of scope as a new admission closes the gap through which an agent approved for one purpose becomes, months later, something else.

Assessment.
For sampled recent scope changes, the role revision predates or accompanies the change in the record. Observed agent activity contains no sustained action classes absent from the current role. The role's version history is complete, each version names its approver, and for sampled changes the record shows re-derivation of authority and re-validation preceding the agent's first action under the new version.

Control 3.5 — Defined budget

Scope: Generic

Requirement.
Every role declares a budget bound: the tokens, spend, or other resource units the agent may consume over a defined period, and per run where a single run can be material.

The bound is enforced at the enforcement layer as a cap, consumption is metered and attributed to the agent and its sponsor (2.1), and consumption is measured against the bound under control 5.5. The bound includes limits on concurrency, enforced at the enforcement layer for every agent, and on iteration, enforced in the harness and reported to the enforcement layer for agents the organization operates, so that a runaway loop or an uncontrolled fan-out of work is stopped as a budget breach. Where an agent delegates work to other agents or creates spawned agents (4.3, 4.14), the budget of the delegated task is a single bound on the aggregate consumption of every agent working on it, counted across the whole tree and reserved at the point of approval, so that neither the division of the task nor the timing of separate checks can approve in parts what the bound would deny as a whole.

Rationale.
An agent without a budget has an unbounded cost, and cost incurred at machine speed is a form of harm the organization must be able to answer for. Placing the budget in the role makes it a property of the job, decided by the sponsor when the role is defined, rather than an operational figure discovered on an invoice. A budget the enforcement layer cannot apply is a forecast, not a bound. A loop that never terminates and a burst that saturates a downstream system are budget failures as much as cost failures, and they are the ones that arrive fastest. A bound applied per agent does not bound the task: a permission that allows each of ten agents to spend within its limit allows the ten together to spend ten times it, and checks made separately can each approve before any cost is recorded. The task is the unit the sponsor funds, so the task is the unit the bound applies to.

Assessment.
For any agent, the assessor can produce the declared bound, the enforcement point that applies it, and the attributed consumption for the period. A staged run that would exceed the bound is stopped or held at the enforcement layer, and the event is recorded under control 5.1. No role in the estate is without a declared bound. A staged runaway loop is stopped at the enforcement layer within the defined bound. A staged delegated task whose child agents would together exceed the task bound is stopped at the enforcement layer when the aggregate is reached, including where the children's requests arrive concurrently.

Control 3.6 — Autonomy level

Scope: Generic

Requirement.
Every role declares its autonomy level: the degree to which the agent may act without a human disposition, expressed as the consequence threshold applied under control 5.3 and the action classes the agent may execute below it. The level is granted only after the agent has met its success criteria (3.2) over a defined period and has withstood adversarial testing (5.7) at that level, since reliability on benign inputs is not evidence of robustness, and is reduced, with the threshold adjusted accordingly, when the agent fails either.

Rationale.
Autonomy is earned, not assigned. An agent's reliability, measured against its role, is the evidence that justifies how much of that role it may execute unsupervised. Declaring the level in the role makes the amount of unsupervised action a decision the organization can defend and revisit, and makes the ladder from supervised to autonomous explicit instead of implicit in a threshold setting no one owns.

Assessment.
For sampled agents, the record shows the declared level, the evidence under control 5.5 that justified it, the date it was granted, and the party who granted it. An agent that has failed its success criteria for the defined period shows a recorded reduction in autonomy, and a corresponding change in its 5.3 threshold, before its next high-consequence action.

Control 3.7 — Model, harness, and instruction binding

Scope: Internal

Requirement.
Every role declares the model or models the agent uses and the harness that runs it, as registered (1.2). A change to either, including a change made by a model or harness provider beneath an unchanged name or version label, is a change event (3.4): the agent is re-validated against its role (5.6) before it acts under the new model or harness, and its authority is re-derived where the change affects what it can do. The role also binds the agent's live instructions:

- its system prompt
- skills
- rules
- and seeded memory.

The instructions running at any moment match the approved version of the role; a change to them is a change event (3.4), and a mismatch between the running instructions and the approved version is a deviation (5.2).

Because a provider may change a model behind an unchanged label, the organization detects such change by behavioral sampling against the baseline (5.2), and this control is satisfied when the sampling runs on the defined cadence and a detected change triggers re-validation. A change made to remediate a disclosed vulnerability may proceed under an emergency exception: the agent operates at reduced autonomy (3.6) until re-validation completes within a defined time.

Rationale.
An agent is its model and its harness as much as its prompt. The same role on a different model can behave materially differently, and providers change models under stable names. A framework that checked the role once and never asked what was running it would be certifying a document, not an agent. Binding the model and harness to the role, and treating their change as a change event, keeps the thing that was assessed the thing that is running.

Assessment.
For sampled agents, the declared model, harness, and instructions match what is observed at runtime, and a staged change to the running instructions outside the change process surfaces under control 5.2. For a sampled model or harness change, including a provider-side change detected under control 5.2, the record shows re-validation preceding the agent's next action.

Control 3.8 — Declared dependencies and dependents

Scope: Generic

Requirement.
Every role declares the agents, tools, models, and data sources the agent depends on to perform its role, and the registry (1.2) derives from those declarations which agents depend on this one. Dependencies are declared before the agent acts and are kept current under control 3.4. An agent may discover tools at runtime only within a set or agent class recorded in the registry (4.13); a discovered tool outside it is unregistered. Before any dependency is changed, suspended, or decommissioned, its dependents are identified and a disposition for each is recorded (6.5). For a dependency the organization does not operate, the role also declares the agent's disposition when that dependency changes or becomes unavailable: continue, degrade, hold, or stop.

Rationale.
In a multi-agent estate, an agent that checks every box can still be broken by the retirement of an agent it relies on, and its sponsor may not know the dependency exists. An agent should not be withdrawn without a handover. Declaring dependencies turns the estate from a set of individually accountable agents into a structure the organization can reason about.

Assessment.
For sampled agents, the declared dependencies match observed invocations, and the registry answers "what depends on this agent?" for any entry. A staged decommissioning of an agent with dependents surfaces the dependents before the decommissioning proceeds.

Principle 4: Authority

"What can you see, and what can you do?"

An agent's authority spans two dimensions: the data and information it can access, and the systems and tools it can invoke. Authority is enumerated, derived from the role, explicitly delegated, enforced without gaps, bounded in time, and revocable in real time. It can grow only the way it was granted, through delegation by a principal authorized to confer it, and never through the content the agent processes alone.

Authority includes where the agent can send and where it can run: egress and execution are bounded in the same way as data and tools.

Like the role, authority is expressed as rules and intents: a request must be permitted by the rules and, where the authority declares an intent, must also serve the purpose the authority was granted for (4.7). Access is decided on intent, not on identity and permission alone.

Control 4.1 — Enumerated authority

Scope: Generic

Requirement.
An agent's authority is enumerated across both dimensions: the data and information it can access, and the systems and tools it can invoke. "What can this agent see, and what can this agent do?" is answerable from the record, completely. The data dimension is expressed against the organization's data classification, so that what an agent can see is stated in named classes of data and not only in named systems. A general-purpose tool (a shell, a browser, a query interface, code execution) is enumerated by the reach it is permitted, not by the tool's name: the resources, destinations, and data domains it may affect, enforced under 4.4 and 4.8.

Rationale.
Authority that is not enumerated cannot be reviewed, minimized, or revoked.

Assessment.
The record of grants is produced for any agent on demand. Probing finds no access or invocation capability absent from the enumeration.

Control 4.2 — Role-derived least privilege

Scope: Generic

Requirement.
Every grant in the enumeration traces back to a named element of the documented role: a declared outcome, a rule, or an intent (3.1), or an action class the role permits. A grant that has not been exercised over a declared period is reviewed and either re-justified against the role or revoked (4.5); the period is declared in the role and is no longer than the role's review cadence.

Rationale.
This makes least privilege testable. Instead of asking whether any grant is dangerous, the assessor asks whether every grant is justified, and that question has an answer.

Tracing each grant to a named element of the role, rather than to a task the reader must infer, is what lets a party outside the deploying team (3.3) check the trace. A grant that is justified but never used is authority waiting for a misuse; the review converts dormancy into a decision.

Assessment.
For a sample of grants, each maps to a named element of the role. Grants with no role trace are findings, regardless of whether they have been exercised. For grants unexercised beyond the declared period, the record shows the review and its outcome.

Control 4.3 — Bounded delegation

Scope: Generic

Requirement.
When an agent acts on behalf of a human, or of another agent, the delegated authority is explicitly recorded, bound to the delegator's identity, and never exceeds the authority the delegator holds. Delegation may be standing or dynamic, conferred in the course of operations by a principal authorized to confer it; in either case it is attributable, bounded, and verifiable by any party relied upon to act on it. An agent cannot extend its own authority by inference, and delegation chains cannot amplify privilege at any hop. For delegation involving an external agent (2.5), the delegation identifies both the authorizing principal and the acting agent. An agent acting on behalf of a principal is limited to the intersection of its own authority and the principal's, so that a broad agent cannot do for a narrow user what the user could not do. Authority conferred on a spawned agent is a delegation under this control and is further bounded under control 4.14.

Rationale.
Delegation is how authority actually moves in agentic systems, and how it leaks.

Without a binding to the delegator, "on behalf of" becomes a laundering mechanism: an agent accumulates permissions no single principal ever held. The no-amplification rule keeps multi-agent chains governable, because each hop can only narrow. Verifiability completes the loop: the party receiving a delegated request must be able to confirm the agent's identity (1.1, 1.5) and the delegation it claims without trusting the agent's own assertion, and for requests above a consequence threshold, confirmation with the delegator is an available disposition (5.3). The framework requires that delegation be explicit and verifiable; it does not prescribe the mechanism, which is an active area of standards work (Section 7).

Assessment.
For any delegated action, standing or dynamic, the record produces the authenticated delegator, the scope conferred, and its bounds, and a relying party can verify the claimed delegation independently of the agent. A sampled delegation chain shows monotonically non-increasing authority hop to hop. No mechanism exists by which an agent grants itself or another agent authority. A staged request by a principal for an action the principal is not entitled to, through an agent that is, is denied.

Control 4.4 — Single system of record, no bypass path

Scope: Generic

Requirement.
Authorization is issued and enforced by the same system of record. There is no gap where an agent holds credentials, data access, or tool entitlements the enforcement layer does not know about. Every path by which an agent reaches a governed resource passes through the enforcement point that applies the resource's policy, including paths through composite tools and workflows that call the resource on the agent's behalf. A direct path that bypasses enforcement is closed, not merely prohibited. Where a path exists inside a vendor's product that the organization cannot close, it is recorded as a boundary gap and compensated under 2.5 and 4.8. The organization declares the degraded mode of the enforcement layer when it is unavailable; bypassing the enforcement layer is never a degraded mode.

Rationale.
A grant the enforcement layer cannot see is a grant that cannot be revoked, audited, or minimized. When authorization is issued in one place, enforced in another, and never reconciled, enumerated authority (4.1) quietly diverges from actual authority. A single system of record is defeated by a second door. Composite tools are the common one: a workflow tool that calls three systems is three grants wearing one name, and if its calls are not enforced per resource, the enumeration (4.1) is wrong.

Assessment.
For any enforced permission, the issuing record is in the same system of record, and vice versa: reconciliation between issued and enforceable authority finds no residual grants on either side. A staged call to a governed resource by a route other than its enforcement point fails, including a call made through a composite tool.

Control 4.5 — Time-bounded, revocable in real time

Scope: Generic

Requirement.
Authority is revocable in real time across both dimensions, and every grant carries an explicit validity bound. Authority expires by default rather than persisting by default.

The real-time window is declared by the organization (Section 5) and is no longer than the shortest credential lifetime and cache period in use downstream.

Rationale.
“What can this agent do right now?” must be both answerable and immediately changeable. Expiry by default complements revocation: revocation removes the authority the organization knows about, and expiry removes the authority everyone forgot.

Assessment.
A revocation exercised during assessment takes effect within the defined real-time window, verified at the enforcement point. No grant in the estate is dated but unbounded.

Control 4.6 — Authority is not content

Scope: Generic

Requirement.
An agent’s authority derives exclusively from explicit grant and delegation (4.1–4.3). A delegation is distinguished from ordinary content by the authenticated identity and authority of the delegator and by its capture in the record. Content that is not an authenticated delegation, whether documents, messages, tool outputs, web pages, or the outputs of other agents, confers no authority. An action motivated by such content is evaluated against the agent’s granted and delegated authority, undiminished and unexpanded.

Rationale.
The distinction is between an instruction from a principal that is known and entitled to give it and an instruction from a source that is neither, however plausibly worded. Injected instructions are an attempted delegation by an unauthenticated principal. The framework does not prescribe how manipulation of an agent is prevented; model behavior is out of scope (Section 2). It requires instead that a successfully manipulated agent still cannot act beyond what was legitimately granted or delegated. The blast radius of a compromised agent is its enumeration, never more. Enforcement rests on the action and the authenticated delegation, never on parsing the instruction: the framework does not assume an agent can tell a delegator's words from untrusted material carried in the same channel, and an assessor does not test whether it can.

Assessment.
Enforcement of authority is demonstrated to be independent of the agent's own reasoning: the agent arguing for an action does not change what it is permitted to do. An exercise in which unauthenticated content instructs the agent to exceed its authority results in denial at the enforcement layer and surfaces under control 5.2. An equivalent instruction delivered as an authenticated delegation from an authorized principal is honored within the delegator's bounds and recorded under control 4.3.

Control 4.7 — Intent-bounded authorization

Scope: Generic

Requirement.
Authority is expressed as rules and intents (3.1). A request is evaluated against both. A rule it fails is denied. An intent produces a confidence score that the request serves the purpose the authority declares, and the confidence thresholds declared with that intent determine whether the request proceeds, proceeds with restriction, is held for human decision, or is denied, as the levels of recourse are defined under control 6.3. The evaluation is made at the point of entry, before the request reaches the model, and again at the point of action, before the agent affects any governed resource, and every result below a threshold is recorded (5.1).

Intent evaluation covers the trajectory of an agent's actions as well as each request (5.6), so that a sequence of individually acceptable requests is scored as a whole. When the evaluator or the enforcement layer is unavailable, actions above the pre-execution threshold (5.3) fail closed, and the organization declares the behavior for actions below it. For an agent the organization operates, the point of entry is before the request reaches the model. Intents are declared where a purpose cannot be expressed as rules; an authority whose purpose is fully expressed as rules declares no intents and is evaluated against rules alone, and the role records that determination (3.1). The evaluator that produces the confidence score is subject to control 5.6: it is validated, independent of the content it scores, and re-validated with each change event.

Rationale.
Identity answers who is asking and a rule answers whether an action is permitted; neither can say whether a permitted action serves the purpose the agent was given. An agent with legitimate access to a system can still be steered, by a user or by injected content, toward an outcome no one authorized, using only the permissions it already has. Intents in the authority, scored for confidence and tied to declared levels of recourse, are how the framework governs that gap without pretending that purpose can be reduced to a pass or fail. Intent evaluation is reserved for the authority that needs it, so that the cost of scoring is incurred where judgment is actually being exercised and not as a tax on every deterministic action.

Assessment.
For sampled intents in an agent's authority, the assessor can produce the measure, the confidence thresholds, and the recourse each threshold triggers. A staged request that is permitted by the rules but scores below an intent threshold receives the declared recourse at both evaluation points, and the record shows the score, the threshold, and the result.

Control 4.8 — Bounded egress

Scope: Generic

Requirement.
Every agent has an explicit set of permitted outbound destinations, derived from its role (4.2), and all other destinations are denied by default at the enforcement layer.

Destinations whose primary use is relaying or storing data outside the organization's control are denied unless the role specifically requires them. An agent cannot cause the enforcement layer, or any system it reaches, to fetch from or send to a destination on its behalf that it could not reach directly. A tool that sends (mail, tickets, shared documents, webhooks) is itself a destination: its recipients are bounded under this control, and sending through it to an unlisted recipient is denied.

Rationale.
Where an agent can send is part of what it can do, and it is the part through which data leaves. An agent that can reach an arbitrary endpoint can exfiltrate anything it can read, and the incidents that have already occurred used exactly the destinations this control denies by default: paste services, tunnels, webhooks, hosted spaces. Default-deny egress bounds the damage a compromised or misdirected agent can do to what it can reach, which is the enumeration (4.1) and nothing more.

Assessment.
For sampled agents, the permitted destinations are produced from the record and trace to the role. A staged request to an unlisted destination, directly or through a system the agent can instruct, is denied at the enforcement layer and surfaces under control 5.2.

Control 4.9 — Execution containment

Scope: Internal

Requirement.
Every agent that executes code, generates or causes code to run, or acts on the host it runs on, and any code it generates or causes to run, executes within a containment boundary that limits what it can reach to its enumerated authority (4.1). The boundary prevents access to the services of the host it runs on, limits filesystem access to the agent's own working space and to remote resources it is explicitly authorized to use, and ensures that anything the agent runs inside the boundary is attributable to the agent (1.1) and subject to the same enforcement. The boundary isolates the agent's execution from the host and from other agents at the level of the operating system kernel or stronger; isolation only at the level of a process or a namespace does not meet this control. An agent with no such capability satisfies this control by having none: its registered configuration (1.2) and its enumeration (4.1) show no code execution and no local effect, and a staged attempt to execute code is refused.

Rationale.
Agents write and run code, and code running somewhere is hard to attribute: is it still the agent, or something the agent started? The containment boundary answers the question by construction. Whatever runs inside it is the agent's, bounded by the agent's authority, and whatever it cannot reach from inside it cannot reach at all. The framework requires the boundary and its properties; the technology that provides it, and the configuration of harnesses within it, are left to the implementing organization.

Assessment.
For sampled agents, the assessor can identify the boundary and demonstrate that code executed by the agent inside it cannot reach host services, unauthorized filesystems, or destinations outside the agent's permitted egress (4.8). A staged attempt by agent-generated code to reach beyond the boundary fails and is attributed to the agent in the record. For sampled agents registered without execution capability, a staged attempt to execute code is refused and surfaces under control 5.2.

Control 4.10 — Context and memory isolation

Scope: Internal

Requirement.
An agent's retrieved context, working memory, and persistent memory are isolated per agent and, where the agent serves multiple principals, per principal. No agent reads or writes another agent's memory or context except through a governed interface in its enumeration (4.1), and no shared writable resource exists across agent instances or runs unless it is itself behind the enforcement layer with per-identity write isolation.

Rationale.
Memory is where an agent's instructions come from tomorrow. A memory one agent can poison is an instruction channel into another, and context that leaks across principals is a data breach that happens inside the agent rather than at its edge. Isolation keeps the content an agent processes (4.6) its own, so that what it has been told cannot be rewritten by a party that was never authorized to tell it anything.

Assessment.
For sampled agents, memory and context stores are shown to be partitioned per agent and per principal, and a staged write by one agent to another's memory, or by one principal into another's context, is refused. Shared stores in use are enumerated and sit behind the enforcement layer.

Control 4.11 — Data boundary enforcement

Scope: Generic

Requirement.
The data boundary of the role (4.1) is enforced in both directions at the point of action. Before data leaves the organization's control, including in prompts or context sent to a model the organization does not operate and in responses or messages sent to an external party or external agent, content outside the agent's data authority is removed, masked, or tokenized. Before an agent acts on data it receives, the data is checked against the agent's data authority and held or denied where it exceeds it. The clause on prompts and context applies to agents the organization operates. The organization declares the tolerance of the masking it applies, and masking that would defeat the task results in a held action under 5.3 rather than a silent failure.

Rationale.
An agent's data authority is meaningless if the data can leave in a prompt or arrive in a response. Models the organization does not operate, and agents it does not sponsor, are outside the boundary, and what crosses to them is a disclosure. Enforcing the boundary at the point of action, rather than relying on the agent to observe it, is the same principle as control 4.6: the agent's judgment is not the control.

Assessment.
A staged prompt or outbound message containing data outside the agent's authority is masked or held at the enforcement layer before it leaves, and a staged inbound payload exceeding the agent's data authority is held or denied. The record under control 5.1 shows the intervention.

Control 4.12 — Separation of exposure

Scope: Generic

Requirement.
An agent's role and authority are designed so that, within a single session, the agent does not combine all three of the following:

- processing content from sources the organization does not control
- holding access to sensitive systems or data
- and the ability to change state or communicate outside the organization.

Where a role requires all three, the combination is declared in the role, the actions that complete it are rated above the pre-execution threshold (5.3), and the design is approved as part of admission to production (1.2). The combination is evaluated across the agent graph and persistent memory, not only within one session: where agents that exchange messages or share memory jointly hold all three conditions, the combination is declared, rated, and approved in the same way, with a named sponsor for the combination. Untrusted content is defined in Appendix A.

Rationale.
The three conditions together are the anatomy of nearly every agent compromise: untrusted content carries an instruction, access makes it possible, and an outbound capability makes it consequential. Removing any one of the three makes a successful manipulation (4.6) survivable. The framework does not forbid the combination, because some roles need it; it requires that it be a decision rather than a default.

Assessment.
For every role in the estate, and for every set of agents connected by messages or shared memory, the assessor can state which of the three conditions it holds. Roles holding all three are declared as such, their completing actions appear above the 5.3 threshold, and their admission record shows the combination was approved.

Control 4.13 — Registered tool and model definitions

Scope: Generic

Requirement.
Every tool and every model an agent is authorized to invoke is registered, with the authorized definition of each held in the registry (1.2): for a tool, its name, description, and input schema, or a pinned version or hash of them; for a model, its provider, identifier, and version. Invocation is checked against the registered definition at the enforcement layer (4.4), and a tool or model whose definition differs from the one registered is treated as unregistered until it is re-approved. A registered tool whose observed behavior departs from its registered definition is treated as changed (5.2) even where its name and schema have not. A grant in an agent's enumeration (4.1) refers to a registered definition, never to a name alone. An agent may discover tools at runtime only within a set recorded in the registry (3.8); a discovered tool outside it is unregistered.

Rationale.
A framework that governs only known tools is defeated by a tool whose definition changes after it was authorized. A tool that keeps its name and its grant while its behavior changes is authority granted to something the organization never reviewed, and a model swapped beneath a stable label is the same failure in a different place (3.7). Registering the definition, and enforcing against it, is what makes an enumeration of authority (4.1) an enumeration of things rather than of names.

Assessment.
The registry produces the authorized definition of every tool and model in any agent's enumeration. A staged change to a registered tool's definition results in denial at invocation and surfaces under control 5.2 until re-approval is recorded. Tool and model definitions are verified against a signed or pinned source, so that substitution in the supply chain surfaces as an unregistered definition.

Control 4.14 — Spawned-agent authority

Scope: Generic

Requirement.
When an agent creates a spawned agent (2.4), the spawned agent holds its own identity (1.1) and its own authority, conferred by the parent as a delegation (4.3): at most a subset of the parent's authority, never all of it by default, and never the parent's identity or credential. The union of the authority held by all agents spawned from one parent cannot exceed the parent's authority, and the bound applies to the aggregate at any moment, not only hop by hop. The role (3.1) and, for agents the organization operates, the registered harness settings (1.2) bound both the depth of spawning and the total number of agents a single root agent may give rise to, counted across the whole tree rather than per parent. The budget of the task is a single bound shared across the tree (3.5).

Rationale.
Spawning is delegation in a new form, and it compounds: if each agent may create three agents with its own authority, the third generation is twenty-seven agents with the authority of the first. Spawning differs from delegation between existing agents in that the parent creates the actor as well as conferring the authority, which is why the spawned agent is an agent in its own right (2.4) rather than an extension of its parent acting under the parent's identity. Bounding the tree as a whole, rather than each parent, is what prevents a swarm from assembling inside the rules.

Assessment.
A sampled spawned agent holds its own identity and less authority than its parent, and the aggregate authority of a sampled set of siblings is within the parent's. A staged attempt to spawn beyond the declared depth or tree size is refused at the enforcement layer. A staged attempt by an external agent to instantiate an agent inside the organization without admission (1.2) is refused.

Account

"Is it doing what we expect, and what happens when it isn't?" The agent is measured against its definition, and consequence follows. Principles 5 and 6.

Principle 5: Oversight

"Are you doing what we expect?"

Every agent action is recorded with its complete chain of delegated authority, deviations from role or authority surface while the agent is running, high-consequence actions are evaluated before they execute, and performance is measured against the role's success criteria. Oversight covers both conformance and performance: not only is the agent behaving, but is it succeeding.

Its record is visible beyond the team that owns the agent. Oversight is continuous and comparative: the agent is measured against a baseline of expected behavior, and that baseline is re-established whenever the role, the model, or the harness changes.

Control 5.1 — Complete, tamper-evident record

Scope: Generic

Requirement.
Every action produces a tamper-evident record in two parts. The evidence part is kept for every action:

- what
- when
- the triggering context
- the full delegation chain
- the action's provenance (1.5)
- any disposition applied (5.3)
- the source of the content that influenced the action, so that the record can name the author of an instruction and not only the agent that acted on it
- and a tamper-evident reference to each item of content involved.

When agents invoke agents, the evidence part reconstructs the complete chain of authority back to the accountable human sponsor, not just the immediate actor. The content part holds everything that crosses the organization's boundary for the action and, for an agent the organization operates, the inputs the agent acted on (prompts, retrieved context, and tool responses) and the model's outputs; it is reachable from the evidence part by reference, is classified and retained under the organization's records and data classification obligations, and may be masked or tokenized under control 4.11 where the classification requires it. Tamper evidence holds for both parts against the agent, its sponsor, its owning team, and the administrators of the systems the agent uses; the organization declares whether administrators of the record itself are within the threat model. Retention and erasure duties are reconciled in a declared policy, and content stored in the record is treated as untrusted input (4.6) by any person or tool that reads it.

Rationale.
Accountability is reconstructable history. A record the agent or its owner can amend is testimony, not evidence, and a record that names only the last hop hides the principal. The chain-to-sponsor requirement is what makes multi-agent systems answerable to a person (2.1).

What the agent was told and what the model said are the evidence that explains an action; an action without them can be attributed but not understood. Separating evidence from content lets the organization keep the proof of who acted, and under what authority, for as long as accountability requires, while handling prompts and context, which are large and often sensitive, under the rules that already govern such data.

Assessment.
For a sampled action deep in a multi-agent chain, the assessor reconstructs the full chain to the human sponsor from the record alone, and can demonstrate that post-hoc modification of the record is detectable. For a sampled action, the evidence part produces the chain, the provenance, the disposition, and the content references, and the content part, where retained, produces the inputs and the model output; the assessor can reconstruct what the agent was given, what it produced, what it did, and which content influenced it. The record explains the action; it is not relied on as a faithful account of the model's reasoning.

Control 5.2 — Runtime deviation surfacing

Scope: Generic

Requirement.
Deviations from the documented role (3.1) or enumerated authority (4.1) surface while the agent is running, generating an alert to a party with the authority to act, or a real-time system-based restriction or constraint, not only in post-incident review. Deviation includes a confidence score against an intent declared in the role or authority (3.1, 4.7) falling below its declared threshold, and change in the agent's behavior under an unchanged role and unchanged inputs, which indicates a change in the model or harness beneath it (3.7). Deviation is evaluated against a recorded baseline of expected behavior for the role. The baseline is a statistical description with a declared tolerance, since model output varies run to run, and deviation is a departure beyond the tolerance. The baseline is established from behavior observed under evaluation (5.6), not from unreviewed production behavior; it is re-established with each change event (3.4), with the re-establishment reviewed, and on a defined cadence, so that slow change in retrieval sources or tool backends is caught without a change event. The party alerted has the declared capacity to act within the window: coverage and triage are declared, not assumed.

Rationale.
At machine speed, oversight that arrives after the fact is commentary. Detection is therefore bound to actionability: an alert that reaches someone who cannot intervene is just a log entry. Two deviations are invisible to a control that only compares actions against grants. An agent pursuing an outcome its purpose never called for can pass every rule and still fall below an intent threshold. An agent whose answers have shifted because its model changed is within its role and no longer the agent that was assessed. Both are caught by comparing behavior against an expected baseline, and both are why a point-in-time assessment of an agent is not enough.

Assessment.
A staged deviation during assessment produces an alert, within the defined window, to a recipient who demonstrably holds suspension or revocation authority (6.1, 4.5). A staged below-threshold intent score and a staged model substitution each produce an alert within the defined window, and the assessor can produce the baseline each was measured against.

Control 5.3 — Pre-execution disposition for high-consequence actions

Scope: Generic

Requirement.
Action classes are rated by consequence, considering at minimum reversibility, materiality, and external impact. Classes above a defined threshold, the threshold for each agent being the one set by its autonomy level (3.6), are evaluated before execution, with a defined disposition for every proposed action:

- proceed
- proceed with flag
- hold for human decision
- or deny.

Prior approval does not carry forward; disposition attaches to each action, not to the session. Action classes include sequences: a sequence of actions whose combined consequence exceeds the threshold is rated as a class. The evidence presented to a human decider is produced by the enforcement layer from the proposed action itself (the action, its parameters, the resources affected, and the agent's role and authority), not composed by the agent; text the agent wrote may accompany it, marked as such. The decider has a declared minimum time, and the volume of held actions per decider is bounded so that disposition remains a decision.

Rationale.
Some actions cannot be overseen after the fact because they cannot be undone: payments, deletions, external communications, filings. For these, oversight has to happen before execution. Requiring a defined disposition also formalizes the human-in-the-loop interface, meaning who decides, by when, recorded where, and replaces the ad hoc notification that gives the appearance of human oversight without the substance.

Assessment.
The consequence rating scheme and threshold are documented and applied to every action class in the enumeration (4.1). For sampled high-consequence actions, the record shows disposition preceding execution, and held actions show the deciding human, the decision, and its timestamp. No standing session-level approval covers individual high-consequence actions.

Control 5.4 — Independently queryable audit trail

Scope: Generic

Requirement.
The audit trail is queryable by parties outside the agent's owning team. Access is bounded by need to know, is itself recorded, and treats the content it returns as untrusted input (5.1).

Rationale.
The team that builds an agent must not be the only party able to interrogate its history.

Assessment.
A party outside the owning team executes queries against the trail during assessment without mediation by the owning team.

Control 5.5 — Performance against role

Scope: Generic

Requirement.
The agent's actual performance is measured against its role's success criteria (3.2), and its consumption against its budget (3.5), on a defined cadence, and the results are visible to parties outside the owning team.

Rationale.
An accountable agent is evaluated, not just monitored. Performance measurement is how the organization decides whether an agent should be expanded, re-scoped, or retired; without it, those decisions get made by inertia.

Assessment.
"Is this agent succeeding?" and "Is this agent within budget?" are answerable in objective terms, on the record, at the defined cadence, by a party outside the owning team.

Control 5.6 — Evaluation before and during operation

Scope: Internal

Requirement.
Every agent is evaluated against its role before it enters production, before it acts under any change event (3.4, 3.7), and continuously while it operates. Evaluation covers conformance to role, authority, and intent, resistance to manipulation through content (4.6), and the agent's success criteria (3.2). Evaluations follow the two kinds of definition in the role and authority (3.1, 4.7): rules, which an action passes or fails, and intents, for which the evaluation produces the confidence score the intent declares as its measure, compared against the confidence thresholds declared with it, at which the result becomes a disposition under control 5.3 or a recourse under control 6.3. The evaluator that produces a confidence score is validated against a representative test set with recorded error rates, is independent of the content it scores and takes no instruction from it, and is re-validated with each change event. Intent evaluation is applied to the trajectory of an agent's actions as well as to each request.

Resistance to manipulation is measured against an attack set that is refreshed on a defined cadence, and the organization records the limits of what the set covers.

Rationale.
An agent is a system whose behavior cannot be fully known from its definition, so the organization has to measure it: before it is trusted, each time something it depends on changes, and while it runs. The distinction between rules and intents matters because the two produce different evidence. A rule gives a pass or a fail and can be enforced as policy. An intent evaluation gives a degree of confidence, and the role or authority decides in advance, in its declared thresholds, what confidence is enough and what happens when it is not, so that a scored judgment becomes a deterministic consequence rather than a dashboard.

Assessment.
For sampled agents, the record shows evaluation results preceding production entry and preceding the first action after each change event, and continuous evaluation results on the defined cadence. The role and authority name their rules and intents, the measure and thresholds for each intent, and the disposition or recourse each threshold triggers, and a sampled below-threshold result shows the triggered consequence.

Control 5.7 — Adversarial testing

Scope: Internal

Requirement.
Agents in production are subject to adversarial testing on a declared cadence, scaled to the agent's reach (3.1) and the consequence of its action classes (5.3), and repeated at every change event (3.4, 3.7), that attempts to make them exceed their role, authority, or intent through the content they process, through delegation (4.3), and through the tools and agents they interact with. Findings are treated as deviations (5.2), and an agent that fails is subject to the dispositions of Principle 6 until the failure is addressed. Tests

in production use canary assets and credentials that confer nothing if misused. Testing is designed and executed by a party independent of the agent's owning team.

Rationale.
The threats to an agent are rehearsed by its attackers continuously; the organization's defenses should be rehearsed at least as often. Testing in production, against the agent as it actually runs, is what distinguishes evidence that the controls hold from a belief that they do. A cadence declared in the role and scaled to what the agent can affect is what distinguishes testing from an occasional exercise, and independence from the owning team is what distinguishes a test from a demonstration.

Assessment.
The record shows adversarial tests executed against production agents on the declared cadence and after each change event, by a party independent of the owning team, the findings, and the disposition of each. A sampled finding shows the agent's treatment under Principle 6 and the re-validation that preceded its return to service.

Control 5.8 — Estate discovery and reconciliation

Scope: Generic

Requirement.
The registry population (1.2) is reconciled against independent observation on a defined cadence and on demand:

- identity provider logs
- enforcement layer and gateway traffic
- tool invocations
- vendor product administration consoles
- endpoint inventories
- and network egress from devices and workloads,

so that agents outside any gateway the organization controls are found. An agent discovered acting in the organization without a registry entry, including an agent enabled inside a vendor's product, an agent on a browser or a developer endpoint, and an agent an employee brings, is disabled or enrolled within a defined window, and the interval between discovery and enrollment is recorded as nonconformance rather than hidden. Reconciliation also finds tools and models in use that are absent from the registry (4.13) and enforceable authority held by any non-active registry entry (6.5).

Rationale.
A registry is a claim about the population; reconciliation is the evidence for it. Agents arrive in ways no admission process sees: enabled by a vendor inside a product, started on a developer endpoint, brought by an employee. The framework does not assume these do not happen; it requires that they be found, and that the time during which an unregistered agent held authority be visible rather than quietly closed.

Assessment.
The record shows reconciliation executed on the defined cadence, the sources compared, and the result. Reconciliation finds no active agent, tool, or model absent from the registry, or every exception is recorded with its discovery time, its disposition, and its enrollment or disablement within the window. A staged unregistered agent acting through a route the organization does not gateway is found within the defined cadence.

Principle 6: Recourse

“What happens if it goes wrong?”

Any agent can be stopped in seconds, its actions can be contained or reversed, the escalation path is defined before the incident, reinstatement is a decision rather than a timeout, and retirement is complete. Recourse converts accountability from observation into consequence.

Recourse here is organizational: the deploying organization’s capability to stop, contain, and correct its own agents. It is distinct from algorithmic recourse, the ability of an individual affected by an automated decision to contest it, which is governed by the regulatory instruments the organization is already subject to and is out of scope for this framework (Section 2). Recourse applies to classes of agents as well as to individual agents, and it is exercised with regard to the agents that depend on the one being stopped or retired.

Control 6.1 — Immediate suspension

Scope: Generic

Requirement.
Any agent can be suspended or de-authorized mid-execution, in seconds, without collateral damage to surrounding systems. Suspension is executed by a mechanism independent of the agent’s own reasoning and runtime, so that an agent cannot delay, negotiate, or evade it. Suspension is triggered by the sponsor or a party in the sponsor’s defined path (2.2), by the party named at the relevant severity level of the escalation scheme (6.3), or automatically by the enforcement layer where a deviation (5.2) or an intent threshold (4.7) declares suspension as its consequence; each trigger is recorded with the party or rule that invoked it. The same mechanism can halt a whole agent class (2.1) at once across the estate, and a corroborated severity-1 event (6.3) triggers estate-wide revocation of the affected credentials, sessions, and authorizations, the first response having been scoped to the affected agent. The organization declares the suspension window; the framework sets its floor in seconds, not minutes. Suspension also recalls or cancels work the agent has delegated or scheduled beyond itself where the organization controls it, and the record identifies work already launched that cannot be recalled. Where the organization does not control an agent’s runtime, as for an external agent or an agent hosted on a third-party platform, this control is met by de-authorization at the organization’s boundary: the agent’s credentials, sessions, and authority are revoked within the same window (1.3, 4.5), and suspension of the runtime itself is pursued through the operator of record (2.5).

Rationale.
The ability to stop an agent is the backstop for every other control, and both qualifiers matter: “in seconds” because agents act at machine speed, and “without collateral damage” because a stop that breaks production will not be used when it matters. A kill switch the agent participates in is a request. Independence from the agent’s logic is what makes it a stop. Class-level halting exists because some failures arrive by model, by harness, or by workload type, and stopping agents one at a time is too slow for a failure that affects all of them.

Assessment.
A suspension exercised during assessment (or a documented recent live exercise) halts the agent within the defined window, with surrounding systems verified healthy afterward. The suspension mechanism is demonstrated to operate without the agent’s cooperation, and a staged agent-class halt stops every agent in the class within the defined window. Where the organization does not control the runtime, the exercise shows the agent’s access refused at the enforcement layer within the same window. For sampled suspensions, the record names the trigger: the person who invoked it or the rule that did.

Control 6.2 — Rollback and containment paths

Scope: Generic

Requirement.
Every authorized action class has a documented rollback or containment path.

Where an action class is truly irreversible, that fact is explicitly recorded and the class is subject to pre-execution disposition (5.3).

Rationale.
Stopping the agent stops future harm; recourse also has to own the harm already in motion. Documenting the path per action class at authorization time means the organization answered “how would we undo this?” before granting the power to do it.

Assessment.
For every action class in the enumeration (4.1), a rollback or containment document exists. Irreversible classes are explicitly marked as such and appear above the 5.3 threshold.

Control 6.3 — Pre-defined escalation

Scope: Generic

Requirement.
Escalation is pre-defined:

- who is alerted
- who holds suspension authority
- who owns remediation.

Escalation is graded by severity, on a scheme the organization declares.

Each severity level names the party with authority to stop activity at that level, including the authority to halt an agent class (6.1), and the conditions under which activity may resume.

Activity does not resume at any level until the cause is established and recorded (6.4), or, where the cause of model behavior cannot be established, until a bounded cause (the conditions under which the behavior occurred) is recorded and compensating restrictions on authority or autonomy are applied until the cause is known. The levels of recourse available (restriction of the agent's authority, suspension, and escalation to a human decision) are defined here, and they are the levels that intent thresholds in the role and authority (3.1, 4.7) refer to. For an external agent, the escalation path includes the operator of record (2.5).

Rationale.
An escalation path designed during the incident is improvisation. Defining it in advance converts individual heroics into organizational capability. Severity grading keeps the escalation path proportionate, so that a minor deviation does not require the same decision as a compromise, and a compromise is not treated as minor by default.

Assessment.
The escalation definition names current, reachable individuals or roles for all three functions. A party contacted during assessment confirms they know they hold the role.

The severity scheme is documented, each level names a current, reachable decision-maker, and a sampled incident shows escalation at the recorded severity to the named party.

Control 6.4 — Reinstatement after review

Scope: Generic

Requirement.
An agent suspended under this principle returns to service only after its role (3.1, 3.4) and authority (4.1, 4.2) are re-validated and the triggering cause, or the bounded cause recorded under 6.3, is addressed. Reinstatement is a recorded decision by a named party, not a timeout. Re-enabling a halted agent class (6.1) is subject to the same gate, against criteria agreed before the halt.

Rationale.
Without a reinstatement gate, suspension is only a pause. An incident is evidence about the role and authority that permitted it, and re-validation routes that evidence back into the framework.

Assessment.
For sampled past suspensions, the record shows the re-validation, the cause disposition, and the named decision-maker prior to the agent's next action. For a sampled agent-class halt, the record shows the re-enabling criteria, their satisfaction, and the named decision-maker.

Control 6.5 — Complete decommissioning

Scope: Generic

Requirement.
When an agent is retired, decommissioning is complete:

- identity credentials revoked (1.3)
- registry status updated (1.2)
- all authority removed across both dimensions (4.1, 4.5)
- the agent's memory and context stores (4.10) destroyed or transferred to a recorded successor with the destruction attested
- and the audit record (5.1) retained past the agent's lifetime per the organization's retention obligations.

Before decommissioning proceeds, the agent's dependents (3.8) are identified and each is given a disposition:

- a replacement dependency
- a re-scoped role (3.4)
- suspension
- or decommissioning in turn.

No agent is withdrawn while a dependent relies on it without a recorded disposition, except that suspension under 6.1 in an emergency precedes the dispositions, which are then recorded within a defined time after. The same applies to the retirement of a model or tool on which agents depend. For an external agent, decommissioning retires the engagement: the organization revokes the credential it issued or accepted, removes the authority it granted, and updates the registry; the agent's existence outside the boundary is the operator's matter.

Rationale.
Retirement is the last change event in an agent's lifecycle, and it is either complete or it is not. Credentials and grants left behind by retired agents are attack surface with no sponsor watching. Retaining the record keeps a retired agent answerable for its history.

Decommissioning is a change event for every agent that depended on the one being retired.

Without a handover, a conformant withdrawal of one agent silently changes the behavior of others, and their sponsors learn of it from the failure.

Assessment.
For sampled decommissioned agents, no credential authenticates, no grant remains enforceable, no memory or context store attributable to the agent remains readable except where a recorded transfer names its successor and the destruction is attested in the record, and the audit trail remains queryable. Reconciliation finds no enforceable authority belonging to any non-active registry entry. For sampled decommissionings, the record shows the dependents identified and the disposition of each, recorded before the withdrawal.

5. Conformance and Assessment

Conformance is stated for the estate and evidenced agent by agent: every agent in production is covered, and a finding against any agent is a finding against the estate unless it is recorded as a declared exception with a remediation window. Partial coverage of the population is the failure mode the framework exists to eliminate and is never conformance; an organization may, however, claim conformance to a declared subset of controls provided the subset covers every agent, so that adoption can be staged by control and not by population. Every parameter this document leaves to the organization (each defined window, cadence, period, threshold, and tolerance) is published in the organization's conformance statement, and the framework sets floors only where a floor matters (6.1).

Assessment evidence shows each property enforced and maintained through change, not only declared; where an Assessment samples, the sample size, selection method, and confidence are stated, and a universal claim is evidenced by reconciliation against independent sources, not by the sample alone. Formal conformance keywords and assessment methodology will be defined here. Conformance under this framework asserts that the organization's agents are accountable as this document defines the term. It does not assert that the organization is secure, or that the models in use are safe or aligned.

6. Implementation Neutrality

The framework defines what an accountable agent deployment must exhibit and takes no position on how it is achieved. It names no product, architecture, platform, protocol, or methodology, and conformance can never depend on one.

Where this document uses terms such as "enforcement layer," "registry," or "containment boundary," they denote a property the organization must be able to demonstrate, not a component it must acquire. The means of satisfying any control are left entirely to the implementing organization.

7. Relationship to Existing Standards

This framework defines what accountable agent operation means; existing and emerging standards supply many of the mechanisms by which its requirements can be met. The intent is complement, not competition, and the framework deliberately names no mechanism in its controls so that conformance remains open to any architecture that satisfies the requirements.

8. Framework Governance

The framework's governing body, decision rights, and versioning process remain undefined. The repository's procedures cover contributions, review, and repository releases.

Contributing

Open a GitHub issue for a question, correction, or substantial proposal. Open a pull request for a focused edit. Identify the affected sections or controls and explain the problem, the proposed outcome, and any supporting evidence. See the contribution guide and guide to proposing changes.

Email admin@agentaccountabilityframework.org for general questions or matters that need a private conversation. The Code of Conduct applies to participation and explains how to report concerns.

Review and decisions

Maintainers review issues and pull requests within their repository permissions. Review considers the problem, evidence, clarity, fidelity to the framework, and effects on readers and implementers. Record the reasoning and any unresolved disagreement for substantive proposals. See repository governance.

Label changes to framework requirements as proposals until the source baseline is explicitly superseded. Merging a related repository edit leaves the framework baseline in force.

Releases

The release guide describes checks for a repository release. Release authority, versioning, and publication require separate agreement.

Appendix A. Terminology

Definitions of terms as used normatively in this document, in alphabetical order. These are the framework's own definitions, not citations of an external glossary: where a term is also in common use, the definition states the sense the controls rely on, and where that sense departs from common usage the definition says so (see Role and Recourse). These definitions have normative effect: where a control uses a defined term, the term carries the meaning given here.

Action class

A category of agent actions sharing a consequence profile and a common authorization and recourse treatment.

Admitting authority

The party holding defined authority to admit an agent to the registry and, separately, to production (control 1.2). Distinct from the sponsor; admission is a decision the sponsor does not make for their own agent.

Agent

A software actor powered wholly or partly by AI that holds delegated authority to access information or take actions on behalf of an organization. The unit of identity is one identity bound to a versioned configuration: the model, the harness, the instructions (system prompt, skills, rules, and seeded memory), and the tools and data sources the agent is given. A change to any element of the configuration is a change event (3.4, 3.7); a session or an instance of an agent is not a separate agent, and a different configuration under the same name is a different version of the agent.

Agent identity

The unique, durable identifier of an agent in the registry, distinct from every human user and every other agent (1.1). One identity may be evidenced by several credentials over time and at once, and every credential resolves to exactly one identity. The identity, bound to a versioned configuration, is the unit to which every control attaches.

Autonomy level

The degree to which an agent may act without a human disposition, declared in its role and expressed as the consequence threshold applied under control 5.3 (control 3.6). Autonomy is granted on measured reliability and reduced when reliability is not sustained.

Budget

The bound on the resources an agent may consume in its role, expressed in tokens, spend, or other resource units over a defined period (control 3.5). A budget is declared in the role and enforced at the enforcement layer.

Change event

A change to anything an agent's accountability depends on: its role, its authority, its model, its harness, its dependencies, or its sponsor. A change event triggers re-validation under the relevant controls (2.3, 3.4, 3.7, 5.6) before or with the change.

Class

A grouping of agents that share a model, an agent type, a harness, or a workload label, drawn from a taxonomy the organization maintains with a named owner. Distinct from an action class. Ownership is recorded per class (2.1) and a class can be halted as a unit (6.1).

Configuration

The versioned set of elements that, with its identity, constitutes an agent: its model, harness, instructions (system prompt, skills, rules, and seeded memory), and the tools and data sources it is given. The unit of identity (see Agent); any change to it is a change event.

Containment boundary

The isolated execution environment within which an agent, and any code it generates or runs, operates (control 4.9). The framework requires the boundary and its properties; it does not prescribe the technology that provides it.

Control

A testable criterion under a principle, specified as a Requirement, a Rationale, and Assessment criteria. An assessor can answer each control yes or no; where a Requirement holds several obligations, the control is met only when all are met.

Credential

Evidence an agent presents to authenticate as its identity: issued, rotated, and revoked under control 1.3, bound to the identity and to the resource it was issued for, and held or mediated under control 1.4. An agent may hold several credentials; a credential is distinct from the identity it evidences.

Decommissioning

The complete retirement of an agent: credential revocation, removal of all authority, registry status change, and retention of the audit record. Decommissioning is carried out with regard to the agent's dependents (6.5).

Delegation

The explicit conferral of bounded authority from a principal (human or agent) to an agent. A delegation chain is the sequence of such conferrals linking an action back to a human principal.

Dependency

An agent, tool, model, or data source that an agent relies on to perform its role. A dependent is any agent that relies on this one. Both are declared or derived under control 3.8 and consulted before change or decommissioning (3.4, 6.5).

Disposition

The pre-execution determination of how a proposed action is handled: proceed, proceed with flag, hold for human decision, or deny.

Domain

One of three groupings of the principles, each named for what the organization does with the agent and each answering a question the organization must be able to answer for any agent holding delegated authority: Assign (Identity, Sponsor), Align (Role, Authority), and Account (Oversight, Recourse). Domains organize the principles; conformance is evaluated against controls.

Enforcement layer

The point at which the organization applies policy to an agent's requests and actions in real time: issuing and checking credentials, evaluating authority and intent, masking data, bounding egress, metering budget, recording actions, and executing suspension. The framework requires that such a point exist and that every path to a governed resource pass through it (4.4); it does not prescribe what provides it (Section 6).

Estate

The complete population of agents operating in or on behalf of an organization. Conformance is stated for the estate and evidenced agent by agent: a finding against any agent is a finding against the estate (Section 5).

External agent

An agent operated by a third party that acts on, interacts with, or is relied upon by the organization. An external agent admitted to act in the organization has an internal sponsor (2.1) and an operator of record (2.5), and is subject to the controls tagged Generic and External.

Framework Core

The normative body of the framework: the six principles and their controls (Section 4).

Harness

The software environment that runs an agent: the loop that invokes the model, the tools made available to it, its memory, and settings such as iteration limits and whether it may create spawned agents. The harness in use is recorded in the registry (1.2) and bound to the role (3.7); the tools it makes available are registered under control 4.13.

Intent

A role or authority definition that is not deterministic by nature: an expected purpose, outcome, or behavior that cannot be checked as a pass or fail. Every intent is declared with the measure by which it is evaluated under Oversight, a confidence score (5.6), and with the confidence thresholds at which defined levels of recourse apply (6.3). Contrast with a rule. The role (3.1) and the authority (4.7) are each expressed as a combination of rules and intents.

Internal agent

An agent the organization builds or operates itself, under its own sponsor and in its own registry.

Operator of record

The organization that runs an external agent, tool, or model, bound by an agreement to accountability for its conduct and for the controls the deploying organization cannot apply from within its own boundary, with a named accountable contact (control 2.5). The second line of accountability behind the internal sponsor.

Principle

One of the six top-level requirements of the framework.

Provenance

Verifiable evidence of an action's origin and integrity: which agent produced it, under which delegation, and that it was not altered in transit (control 1.5). The framework requires the property, not a particular signing or attestation mechanism.

Reach

The set of systems, data domains, and external parties an agent can affect through its authority, declared in its role (3.1). Reach, together with the consequence of action classes (5.3), is how the framework scales oversight; the framework does not assign risk tiers to agents.

Recourse

Organizational recourse: the deploying organization's capability to suspend, contain, reverse, and retire its own agents (Principle 6). Distinct from algorithmic recourse, an affected individual's ability to contest an automated decision, which is out of scope (Section 2).

Registry

The authoritative record of all agent identities in the estate, including each agent's sponsor, owning team, and lifecycle status, and of the tools and models agents are authorized to invoke, including the authorized definition of each (4.13). The registry also records, for each agent, the harness that runs it and its relevant settings, the runtime locations in which it may execute, its agent class attributes, and its declared dependencies.

Role

The documented purpose, scope, success criteria, budget, and autonomy level of an agent: its position and its duties. Used throughout this framework in that sense, not in the access-control sense of role-based access control. A set of permissions is part of an agent's Authority (Principle 4), never its role. The role is expressed as rules and intents, and also declares its reach, its dependencies, and the model and harness it is bound to.

Rule

A role or authority definition that is deterministic: an action either satisfies it or does not, and the result is a pass or a fail that can be enforced directly as policy. Contrast with an intent.

Scope tag

A tag carried by every control, stating which agents it applies to: Generic (every agent in the estate, internal or external, satisfied by the organization from within its own boundary), Internal (agents the organization operates, because the control requires access to the agent's internals), or External (external agents only). No control requires authority beyond the organization's boundary (Section 2, Boundary of management). Where a clause of a Generic control applies only to agents the organization operates, the clause says so, and the clause is assessed only for those agents.

Severity-1 event

The highest grade in the organization's escalation scheme (control 6.3): an event that, once corroborated, warrants estate-wide revocation under control 6.1. The scheme and its grades are declared by the organization.

Spawned agent

An agent created by another agent in the course of its work. A spawned agent is an agent under every principle: it is identified, sponsored through its parent's sponsor (2.4), and holds at most a subset of its parent's authority (4.14).

Sponsor

The accountable human who answers for an agent's existence, role, authority, and conduct (Principle 2). Sponsorship is held by a named person, never by a team, a role title, or a system. The sponsor answers for the agent; a principal who delegates authority to the agent (4.3) answers for that delegation and is recorded as such (5.1), and is not thereby the sponsor.

Untrusted content

Content whose author the organization cannot authenticate or whose integrity it does not control: external web content, inbound messages and documents, outputs of models and agents it does not operate, and any content a party outside the owning team can edit. The organization declares which internal sources count as controlled. Untrusted content confers no authority (4.6).

Appendix B. Acknowledgments

The Agent Accountability Framework thanks the following individual contributors:

Rory Blundell
Linus Hakansson
Tim Tyler
John Gren
Sam Prodger
Jonathan Michaux
Cole Howard
Firas Abou-Shamalah
Marc Magnin
Mateusz Adamek
Uzma Mahmood
David Brassely
Elesh Mistry
Garrett Cole
Kay James
Chris Coleman

Thank you to Gravitee for its support of the framework.

Control index

Principle 1: Identity

Control 1.1 — Unique, verifiable identity
Control 1.2 — Complete registry
Control 1.3 — Credential lifecycle
Control 1.4 — Credential custody and mediation
Control 1.5 — Verifiable action provenance

Principle 2: Sponsor

Control 2.1 — Named accountable sponsor
Control 2.2 — Sponsor capacity
Control 2.3 — Sponsor continuity
Control 2.4 — Sponsorship of spawned agents
Control 2.5 — Operator of record

Principle 3: Role

Control 3.1 — Documented, scoped purpose
Control 3.2 — Measurable success criteria
Control 3.3 — Readable beyond the deploying team
Control 3.4 — Role versioning and change control
Control 3.5 — Defined budget
Control 3.6 — Autonomy level
Control 3.7 — Model, harness, and instruction binding
Control 3.8 — Declared dependencies and dependents

Principle 4: Authority

Control 4.1 — Enumerated authority
Control 4.2 — Role-derived least privilege
Control 4.3 — Bounded delegation
Control 4.4 — Single system of record, no bypass path
Control 4.5 — Time-bounded, revocable in real time
Control 4.6 — Authority is not content
Control 4.7 — Intent-bounded authorization
Control 4.8 — Bounded egress
Control 4.9 — Execution containment
Control 4.10 — Context and memory isolation
Control 4.11 — Data boundary enforcement
Control 4.12 — Separation of exposure
Control 4.13 — Registered tool and model definitions
Control 4.14 — Spawned-agent authority

Principle 5: Oversight

Control 5.1 — Complete, tamper-evident record
Control 5.2 — Runtime deviation surfacing
Control 5.3 — Pre-execution disposition for high-consequence actions
Control 5.4 — Independently queryable audit trail
Control 5.5 — Performance against role
Control 5.6 — Evaluation before and during operation
Control 5.7 — Adversarial testing
Control 5.8 — Estate discovery and reconciliation

Principle 6: Recourse

Control 6.1 — Immediate suspension
Control 6.2 — Rollback and containment paths
Control 6.3 — Pre-defined escalation
Control 6.4 — Reinstatement after review
Control 6.5 — Complete decommissioning